

ig - SYNTAX ERROR

St x i v e n

a 2 z

D S A S H E E T

- " 4 9 5 "

By - SYNTAX ERROR

Step 7: Bit Manipulation

The PDF created by Abhishek Rathor (Instagram:SYNTAX ERROR)

PROBLEM 1: Check if the i-th bit is set or not

Problem Statement:

Write a Java program to check if the i-th bit (0-indexed from the right) of a number is set (1) or not (0).

Java Code:

```
import java.util.Scanner;

public class CheckIthBit {
    public static boolean isBitSet(int n, int i) {
        return (n & (1 << i)) != 0;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number: ");
        int n = scanner.nextInt();
        System.out.print("Enter bit position: ");
        int i = scanner.nextInt();
        System.out.println("Is " + i + "-th bit set? " + isBitSet(n, i));
        scanner.close();
    }
}
```

Explanation:

1. Shift 1 to the left by i positions.
 2. Use bitwise AND with the number. If result is non-zero, the bit is set.
-

PROBLEM 2: Check if a number is odd or not

Problem Statement:

Write a Java program to check if a number is odd using bit manipulation.

Java Code:

```
import java.util.Scanner;
```

```

public class CheckOdd {
    public static boolean isOdd(int n) {
        return (n & 1) == 1;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number: ");
        int n = scanner.nextInt();
        System.out.println(n + " is odd? " + isOdd(n));
        scanner.close();
    }
}

```

Explanation:

1. The least significant bit of an odd number is 1.
 2. Use bitwise AND with 1 to check it.
-

PROBLEM 3: Check if a number is power of 2 or not

Problem Statement:

Write a Java program to check if a number is a power of 2 using bit manipulation.

Java Code:

```

import java.util.Scanner;

public class PowerOfTwo {
    public static boolean isPowerOfTwo(int n) {
        return n > 0 && (n & (n - 1)) == 0;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number: ");
        int n = scanner.nextInt();
        System.out.println(n + " is power of 2? " + isPowerOfTwo(n));
        scanner.close();
    }
}

```

Explanation:

1. Power of 2 has only one bit set.
 2. $n \& (n-1)$ removes the lowest set bit. If result is 0, it's a power of 2.
-

PROBLEM 4: Count the number of set bits

Problem Statement:

Write a Java program to count the number of 1s in the binary representation of a number.

Java Code:

```
import java.util.Scanner;

public class CountSetBits {
    public static int countBits(int n) {
        int count = 0;
        while (n != 0) {
            count += (n & 1);
            n >>= 1;
        }
        return count;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number: ");
        int n = scanner.nextInt();
        System.out.println("Number of set bits: " + countBits(n));
        scanner.close();
    }
}
```

Explanation:

1. Check the last bit using $n \& 1$.
 2. Right shift the number and count until n becomes 0.
-

PROBLEM 5: Set/Unset the rightmost unset bit

Problem Statement:

Write a Java program to set the rightmost unset bit of a number.

Java Code:

```
import java.util.Scanner;

public class SetRightmostUnsetBit {
    public static int setRightmostUnsetBit(int n) {
        return n | (n + 1);
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
    }
```

```

        System.out.print("Enter number: ");
        int n = scanner.nextInt();
        System.out.println("After setting rightmost unset bit: " +
setRightmostUnsetBit(n));
        scanner.close();
    }
}

```

Explanation:

1. Adding 1 flips the rightmost unset bit and sets it.
 2. OR operation keeps other bits unchanged and sets the bit.
-

PROBLEM 6: Swap two numbers

Problem Statement:

Write a Java program to swap two numbers using XOR (no temp variable).

Java Code:

```

import java.util.Scanner;

public class SwapXOR {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter first number (a): ");
        int a = scanner.nextInt();
        System.out.print("Enter second number (b): ");
        int b = scanner.nextInt();

        a = a ^ b;
        b = a ^ b;
        a = a ^ b;

        System.out.println("After swap: a = " + a + ", b = " + b);
        scanner.close();
    }
}

```

Explanation:

1. XORing a number with another twice cancels it.
 2. This lets you swap without using extra space.
-

***PROBLEM 7: Divide two integers without using , / or %**

Problem Statement:

Write a Java program to divide two integers without using multiplication, division, or modulo operators.

Java Code:

```
import java.util.Scanner;

public class DivideWithoutOperators {
    public static int divide(int dividend, int divisor) {
        if (dividend == Integer.MIN_VALUE && divisor == -1)
            return Integer.MAX_VALUE;

        int sign = (dividend > 0) == (divisor > 0) ? 1 : -1;
        long a = Math.abs((long) dividend);
        long b = Math.abs((long) divisor);
        int result = 0;

        while (a >= b) {
            long temp = b, multiple = 1;
            while (a >= (temp << 1)) {
                temp <<= 1;
                multiple <<= 1;
            }
            a -= temp;
            result += multiple;
        }

        return sign * result;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter dividend: ");
        int dividend = scanner.nextInt();
        System.out.print("Enter divisor: ");
        int divisor = scanner.nextInt();
        System.out.println("Quotient: " + divide(dividend, divisor));
        scanner.close();
    }
}
```

Explanation:

1. Use bitwise left shifts to multiply divisor and subtract in chunks.
 2. Track how many times it fits and accumulate the result.
 3. This approach works in logarithmic time.
-

PROBLEM 8: Count number of bits to be flipped to convert A to B

Problem Statement:

Write a Java program to count the number of bits that need to be flipped to convert integer A to integer B.

Java Code:

```
java
CopyEdit
import java.util.Scanner;

public class BitFlipCounter {
    public static int countFlips(int a, int b) {
        int xor = a ^ b;
        int count = 0;
        while (xor != 0) {
            count += (xor & 1);
            xor >>= 1;
        }
        return count;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number A: ");
        int a = sc.nextInt();
        System.out.print("Enter number B: ");
        int b = sc.nextInt();
        System.out.println("Bits to flip: " + countFlips(a, b));
        sc.close();
    }
}
```

Explanation:

1. XOR of A and B gives bits that are different.
 2. Count number of 1s in the XOR result.
-

PROBLEM 9: Find the number that appears odd number of times

Problem Statement:

Write a Java program to find the number that appears an odd number of times in an array. Only one such number exists.

Java Code:

```
java
CopyEdit
public class OddOccurrence {
    public static int findOdd(int[] arr) {
        int res = 0;
    }
}
```

```

        for (int num : arr) {
            res ^= num;
        }
        return res;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 2, 3, 1, 3};
        System.out.println("Number occurring odd times: " + findOdd(arr));
    }
}

```

Explanation:

1. XOR of a number with itself is 0.
 2. XOR all numbers — even pairs cancel out, the odd occurrence remains.
-

PROBLEM 10: Power Set

Problem Statement:

Write a Java program to generate the power set (all subsets) of a given string.

Java Code:

```

java
CopyEdit
public class PowerSet {
    public static void generateSubsets(String s) {
        int n = s.length();
        int total = 1 << n; // 2^n subsets
        for (int i = 0; i < total; i++) {
            StringBuilder subset = new StringBuilder();
            for (int j = 0; j < n; j++) {
                if ((i & (1 << j)) != 0)
                    subset.append(s.charAt(j));
            }
            System.out.println(subset);
        }
    }

    public static void main(String[] args) {
        String s = "abc";
        System.out.println("Power Set:");
        generateSubsets(s);
    }
}

```

Explanation:

1. Every number from 0 to $2^n - 1$ represents a subset.
2. Use bits of each number to decide inclusion of characters.

PROBLEM 11: Find XOR of numbers from L to R

Problem Statement:

Write a Java program to find the XOR of all numbers from L to R (inclusive).

Java Code:

```
java
CopyEdit
public class RangeXOR {
    public static int xorTill(int x) {
        if (x % 4 == 0) return x;
        if (x % 4 == 1) return 1;
        if (x % 4 == 2) return x + 1;
        return 0;
    }

    public static int xorRange(int L, int R) {
        return xorTill(R) ^ xorTill(L - 1);
    }

    public static void main(String[] args) {
        int L = 3, R = 9;
        System.out.println("XOR from " + L + " to " + R + " is: " +
xorRange(L, R));
    }
}
```

Explanation:

1. XOR from 0 to x follows a pattern:
 - $x \% 4 == 0 \rightarrow x$
 - $x \% 4 == 1 \rightarrow 1$
 - $x \% 4 == 2 \rightarrow x + 1$
 - $x \% 4 == 3 \rightarrow 0$
 2. XOR from L to R = XOR(0 to R) ^ XOR(0 to L-1)
-

PROBLEM 12: Find the two numbers appearing odd number of times

Problem Statement:

Write a Java program to find the two numbers in an array that appear an odd number of times. All others appear even times.

Java Code:

```
java
CopyEdit
```

```

public class TwoOddOccurrences {
    public static void findTwoOdd(int[] arr) {
        int xor = 0;
        for (int num : arr) xor ^= num;

        int rightmostBit = xor & -xor;
        int x = 0, y = 0;
        for (int num : arr) {
            if ((num & rightmostBit) != 0)
                x ^= num;
            else
                y ^= num;
        }
        System.out.println("Two odd occurring numbers are: " + x + " and "
+ y);
    }

    public static void main(String[] args) {
        int[] arr = {4, 3, 4, 4, 4, 5, 3, 5, 7, 9};
        findTwoOdd(arr);
    }
}

```

Explanation:

1. XOR all elements — result is $a \oplus b$.
2. Find rightmost set bit to divide numbers into 2 groups.
3. XOR separately to get the two odd appearing numbers.

PROBLEM 13: Print Prime Factors of a Number

Problem Statement:

Write a Java program to print all prime factors of a given number.

Java Code:

```

java
CopyEdit
public class PrimeFactors {
    public static void printPrimeFactors(int n) {
        while (n % 2 == 0) {
            System.out.print(2 + " ");
            n /= 2;
        }
        for (int i = 3; i <= Math.sqrt(n); i += 2) {
            while (n % i == 0) {
                System.out.print(i + " ");
                n /= i;
            }
        }
        if (n > 2) {
            System.out.print(n);
        }
    }
}

```

```

    public static void main(String[] args) {
        int n = 315;
        System.out.print("Prime factors: ");
        printPrimeFactors(n);
    }
}

```

Explanation:

1. Divide by 2 until odd.
 2. Try odd divisors from 3 up to $\sqrt{n}$.
 3. If $n > 2$, it is a prime.
-

PROBLEM 14: All Divisors of a Number

Problem Statement:

Write a Java program to print all divisors of a given number in sorted order.

Java Code:

```

java
CopyEdit
import java.util.*;

public class AllDivisors {
    public static void printDivisors(int n) {
        ArrayList<Integer> large = new ArrayList<>();
        for (int i = 1; i * i <= n; i++) {
            if (n % i == 0) {
                System.out.print(i + " ");
                if (i != n / i) {
                    large.add(n / i);
                }
            }
        }
        Collections.reverse(large);
        for (int val : large) System.out.print(val + " ");
    }

    public static void main(String[] args) {
        int n = 36;
        System.out.print("Divisors: ");
        printDivisors(n);
    }
}

```

Explanation:

1. Loop from 1 to $\sqrt{n}$.
2. Store larger divisors and reverse them to maintain sorted order.

PROBLEM 15: Sieve of Eratosthenes

Problem Statement:

Write a Java program to print all prime numbers up to n using the Sieve of Eratosthenes.

Java Code:

```
java
CopyEdit
public class Sieve {
    public static void sieve(int n) {
        boolean[] prime = new boolean[n + 1];
        for (int i = 2; i <= n; i++) prime[i] = true;

        for (int p = 2; p * p <= n; p++) {
            if (prime[p]) {
                for (int i = p * p; i <= n; i += p) {
                    prime[i] = false;
                }
            }
        }

        for (int i = 2; i <= n; i++) {
            if (prime[i]) System.out.print(i + " ");
        }

        public static void main(String[] args) {
            int n = 50;
            System.out.print("Primes up to " + n + ": ");
            sieve(n);
        }
    }
}
```

Explanation:

1. Mark all numbers prime initially.
2. Mark multiples of each prime as false.
3. Print the remaining primes.

PROBLEM 16: Find Prime Factorisation using Sieve

Problem Statement:

Write a Java program to precompute smallest prime factors using Sieve and use it to find prime factorization.

Java Code:

```
java
CopyEdit
public class SieveFactorization {
    static int[] spf = new int[10001];

    public static void sieve() {
        for (int i = 2; i < spf.length; i++) spf[i] = i;

        for (int i = 2; i * i < spf.length; i++) {
            if (spf[i] == i) {
                for (int j = i * i; j < spf.length; j += i) {
                    if (spf[j] == j) spf[j] = i;
                }
            }
        }
    }

    public static void factorize(int n) {
        while (n != 1) {
            System.out.print(spf[n] + " ");
            n /= spf[n];
        }
    }

    public static void main(String[] args) {
        sieve();
        int n = 100;
        System.out.print("Prime factors of " + n + ": ");
        factorize(n);
    }
}
```

Explanation:

1. Precompute smallest prime factor (SPF) for every number.
 2. To factor a number, divide by its SPF repeatedly.
-

PROBLEM 17: Power(n, x)

Problem Statement:

Write a Java program to compute n^x using binary exponentiation in $O(\log x)$ time.

Java Code:

```
java
CopyEdit
public class FastPower {
    public static long power(long n, int x) {
        long result = 1;
        while (x > 0) {
            if ((x & 1) == 1)
                result *= n;
        }
    }
}
```

```

        n *= n;
        x >>= 1;
    }
    return result;
}

public static void main(String[] args) {
    int n = 2, x = 10;
    System.out.println(n + "^" + x + " = " + power(n, x));
}
}

```

Explanation:

1. If power is odd, multiply result by current base.
2. Square the base and halve the exponent.
3. Repeat until exponent is 0.

Step 9: Stack queue

PROBLEM 18: Implement Stack using Arrays

Problem Statement:

Write a Java program to implement a stack using arrays with basic operations: push, pop, peek, and isEmpty.

Java Code:

```

public class ArrayStack {
    private int top;
    private int[] stack;
    private int capacity;

    public ArrayStack(int size) {
        stack = new int[size];
        capacity = size;
        top = -1;
    }

    public void push(int x) {
        if (top == capacity - 1) {
            System.out.println("Stack Overflow");
            return;
        }
        stack[++top] = x;
    }

    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack Underflow");
            return -1;
        }
    }
}

```

```

        return stack[top--];
    }

    public int peek() {
        if (isEmpty()) return -1;
        return stack[top];
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public static void main(String[] args) {
        ArrayStack s = new ArrayStack(5);
        s.push(10);
        s.push(20);
        System.out.println(s.pop());
        System.out.println(s.peek());
    }
}

```

Explanation:

1. The stack is backed by an integer array.
 2. Operations `push`, `pop`, `peek`, and `isEmpty` mimic standard stack behavior.
-

 **Time Complexity: $O(N \log M)$.**

PROBLEM 19: Implement Queue using Arrays

Problem Statement:

Write a Java program to implement a queue using arrays with basic operations: `enqueue`, `dequeue`, `peek`, and `isEmpty`.

Java Code:

```

java
CopyEdit
public class ArrayQueue {
    private int front, rear, size;
    private int[] queue;

    public ArrayQueue(int capacity) {
        queue = new int[capacity];
        front = 0;
        rear = -1;
    }
}

```

```

        size = 0;
    }

    public void enqueue(int x) {
        if (size == queue.length) {
            System.out.println("Queue Full");
            return;
        }
        rear = (rear + 1) % queue.length;
        queue[rear] = x;
        size++;
    }

    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue Empty");
            return -1;
        }
        int val = queue[front];
        front = (front + 1) % queue.length;
        size--;
        return val;
    }

    public int peek() {
        return isEmpty() ? -1 : queue[front];
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public static void main(String[] args) {
        ArrayQueue q = new ArrayQueue(5);
        q.enqueue(10);
        q.enqueue(20);
        System.out.println(q.dequeue());
        System.out.println(q.peek());
    }
}

```

Explanation:

1. Queue is implemented using a circular array.
 2. enqueue and dequeue operate in $O(1)$ time with modulo indexing.
-

PROBLEM 20: Implement Stack using Queue

Problem Statement:

Write a Java program to implement a stack using two queues (push, pop, peek, isEmpty).

Java Code:

```
import java.util.LinkedList;
import java.util.Queue;

public class StackUsingQueue {
    Queue<Integer> q1 = new LinkedList<>();
    Queue<Integer> q2 = new LinkedList<>();

    public void push(int x) {
        q2.add(x);
        while (!q1.isEmpty()) {
            q2.add(q1.remove());
        }
        Queue<Integer> temp = q1;
        q1 = q2;
        q2 = temp;
    }

    public int pop() {
        if (q1.isEmpty()) return -1;
        return q1.remove();
    }

    public int peek() {
        if (q1.isEmpty()) return -1;
        return q1.peek();
    }

    public boolean isEmpty() {
        return q1.isEmpty();
    }

    public static void main(String[] args) {
        StackUsingQueue s = new StackUsingQueue();
        s.push(10);
        s.push(20);
        System.out.println(s.pop());
        System.out.println(s.peek());
    }
}
```

Explanation:

1. push puts the new element into an empty queue and moves all old elements behind it.
 2. pop and peek work on the front of q1.
-

PROBLEM 21: Implement Queue using Stack

Problem Statement:

Write a Java program to implement a queue using two stacks (enqueue, dequeue, peek, isEmpty).

Java Code:

```
import java.util.Stack;

public class QueueUsingStack {
    Stack<Integer> s1 = new Stack<>();
    Stack<Integer> s2 = new Stack<>();

    public void enqueue(int x) {
        s1.push(x);
    }

    public int dequeue() {
        if (isEmpty()) return -1;
        if (s2.isEmpty()) {
            while (!s1.isEmpty()) s2.push(s1.pop());
        }
        return s2.pop();
    }

    public int peek() {
        if (isEmpty()) return -1;
        if (s2.isEmpty()) {
            while (!s1.isEmpty()) s2.push(s1.pop());
        }
        return s2.peek();
    }

    public boolean isEmpty() {
        return s1.isEmpty() && s2.isEmpty();
    }

    public static void main(String[] args) {
        QueueUsingStack q = new QueueUsingStack();
        q.enqueue(10);
        q.enqueue(20);
        System.out.println(q.dequeue());
        System.out.println(q.peek());
    }
}
```

Explanation:

1. enqueue pushes into s1.
 2. dequeue transfers elements from s1 to s2 if s2 is empty, then pops from s2.
-

PROBLEM 22: Implement Stack using Linked List

Problem Statement:

Write a Java program to implement a stack using a linked list (push, pop, peek, isEmpty).

Java Code:

```
class Node {
    int data;
    Node next;
    Node(int d) {
        data = d;
        next = null;
    }
}

public class StackLL {
    Node top = null;

    public void push(int x) {
        Node temp = new Node(x);
        temp.next = top;
        top = temp;
    }

    public int pop() {
        if (isEmpty()) return -1;
        int val = top.data;
        top = top.next;
        return val;
    }

    public int peek() {
        return isEmpty() ? -1 : top.data;
    }

    public boolean isEmpty() {
        return top == null;
    }

    public static void main(String[] args) {
        StackLL s = new StackLL();
        s.push(10);
        s.push(20);
        System.out.println(s.pop());
        System.out.println(s.peek());
    }
}
```

Explanation:

1. Stack uses LIFO principle via a linked list.
 2. push adds to the head, pop removes from the head.
-

PROBLEM 23: Implement Queue using Linked List

Problem Statement:

Write a Java program to implement a queue using a linked list (enqueue, dequeue, peek, isEmpty).

Java Code:

```
class QNode {
    int data;
    QNode next;
    QNode(int d) {
        data = d;
        next = null;
    }
}

public class QueueLL {
    QNode front = null, rear = null;

    public void enqueue(int x) {
        QNode temp = new QNode(x);
        if (rear == null) {
            front = rear = temp;
            return;
        }
        rear.next = temp;
        rear = temp;
    }

    public int dequeue() {
        if (isEmpty()) return -1;
        int val = front.data;
        front = front.next;
        if (front == null) rear = null;
        return val;
    }

    public int peek() {
        return isEmpty() ? -1 : front.data;
    }

    public boolean isEmpty() {
        return front == null;
    }

    public static void main(String[] args) {
        QueueLL q = new QueueLL();
        q.enqueue(10);
        q.enqueue(20);
        System.out.println(q.dequeue());
        System.out.println(q.peek());
    }
}
```

Explanation:

1. Queue uses FIFO principle via linked list.
 2. enqueue inserts at rear, dequeue removes from front.
-

PROBLEM 24: Check for Balanced Parenthesis

Problem Statement:

Write a Java program to check if the given expression has balanced parentheses.

Java Code:

```
import java.util.Stack;

public class BalancedParenthesis {
    public static boolean isBalanced(String s) {
        Stack<Character> st = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') {
                st.push(c);
            } else {
                if (st.isEmpty()) return false;
                char top = st.pop();
                if ((c == ')' && top != '(') ||
                    (c == '}' && top != '{') ||
                    (c == ']' && top != '['))
                    return false;
            }
        }
        return st.isEmpty();
    }

    public static void main(String[] args) {
        String expr = "{[()]}" ;
        System.out.println(isBalanced(expr));
    }
}
```

Explanation:

1. A stack is used to keep track of opening brackets.
 2. Each closing bracket must match the latest opening bracket.
-

PROBLEM 25: Implement Min Stack

Problem Statement:

Write a Java program to design a stack that supports push, pop, top, and getMin in constant time.

Java Code:

```
import java.util.Stack;

public class MinStack {
    Stack<Integer> stack = new Stack<>();
    Stack<Integer> minStack = new Stack<>();
}
```

```

public void push(int x) {
    stack.push(x);
    if (minStack.isEmpty() || x <= minStack.peek())
        minStack.push(x);
}

public void pop() {
    if (stack.peek().equals(minStack.peek()))
        minStack.pop();
    stack.pop();
}

public int top() {
    return stack.peek();
}

public int getMin() {
    return minStack.peek();
}

public static void main(String[] args) {
    MinStack ms = new MinStack();
    ms.push(5);
    ms.push(2);
    ms.push(8);
    ms.pop();
    System.out.println(ms.getMin());
}
}

```

Explanation:

1. Maintain a secondary stack to store minimums.
 2. Ensures `getMin()` works in $O(1)$ time.
-

PROBLEM: 26

Problem Statement:

Write a Java program to convert an Infix expression to Postfix using Stack.

Java Code:

```

java
CopyEdit
import java.util.*;

public class InfixToPostfix {
    public static int precedence(char ch) {
        switch (ch) {
            case '+': case '-': return 1;
            case '*': case '/': return 2;

```

```

        case '^': return 3;
    }
    return -1;
}

public static String convert(String exp) {
    StringBuilder result = new StringBuilder();
    Stack<Character> stack = new Stack<>();

    for (int i = 0; i < exp.length(); i++) {
        char c = exp.charAt(i);

        if (Character.isLetterOrDigit(c)) {
            result.append(c);
        } else if (c == '(') {
            stack.push(c);
        } else if (c == ')') {
            while (!stack.isEmpty() && stack.peek() != '(')
                result.append(stack.pop());
            stack.pop();
        } else {
            while (!stack.isEmpty() && precedence(c) <=
precedence(stack.peek()))
                result.append(stack.pop());
            stack.push(c);
        }
    }

    while (!stack.isEmpty())
        result.append(stack.pop());

    return result.toString();
}

public static void main(String[] args) {
    String infix = "a+b*(c^d-e)^(f+g*h)-i";
    System.out.println("Postfix: " + convert(infix));
}
}

```

Explanation:

1. Traverse the infix expression.
 2. Add operands to the result.
 3. Use stack to maintain operator precedence and handle parentheses.
 4. Pop any remaining operators to result.
-

PROBLEM: 27

Problem Statement:

Convert a Prefix expression to Infix using Stack.

Java Code:

```
java
CopyEdit
import java.util.*;

public class PrefixToInfix {
    public static String convert(String expr) {
        Stack<String> stack = new Stack<>();
        for (int i = expr.length() - 1; i >= 0; i--) {
            char c = expr.charAt(i);
            if (Character.isLetterOrDigit(c)) {
                stack.push(c + "");
            } else {
                String op1 = stack.pop();
                String op2 = stack.pop();
                stack.push("(" + op1 + c + op2 + "");
            }
        }
        return stack.peek();
    }

    public static void main(String[] args) {
        String prefix = "+AB-CD";
        System.out.println("Infix: " + convert(prefix));
    }
}
```

Explanation:

1. Traverse from right to left.
 2. Push operands.
 3. On operator, pop two operands, combine, and push back as infix.
-

PROBLEM: 28

Problem Statement:

Convert a Prefix expression to Postfix using Stack.

Java Code:

```
java
CopyEdit
import java.util.*;

public class PrefixToPostfix {
    public static String convert(String expr) {
        Stack<String> stack = new Stack<>();
        for (int i = expr.length() - 1; i >= 0; i--) {
            char c = expr.charAt(i);
            if (Character.isLetterOrDigit(c)) {
```

```

        stack.push(c + "");
    } else {
        String op1 = stack.pop();
        String op2 = stack.pop();
        stack.push(op1 + op2 + c);
    }
}
return stack.peek();
}

public static void main(String[] args) {
    String prefix = "+*AB-CD";
    System.out.println("Postfix: " + convert(prefix));
}
}

```

Explanation:

1. Traverse the prefix string in reverse.
 2. Push operands.
 3. On operator, pop two, combine in postfix, and push back.
-

PROBLEM: 29

Problem Statement:

Convert Postfix to Prefix using Stack.

Java Code:

```

java
CopyEdit
import java.util.*;

public class PostfixToPrefix {
    public static String convert(String expr) {
        Stack<String> stack = new Stack<>();
        for (int i = 0; i < expr.length(); i++) {
            char c = expr.charAt(i);
            if (Character.isLetterOrDigit(c)) {
                stack.push(c + "");
            } else {
                String op2 = stack.pop();
                String op1 = stack.pop();
                stack.push(c + op1 + op2);
            }
        }
        return stack.peek();
    }

    public static void main(String[] args) {
        String postfix = "AB+CD-*";
    }
}

```

```
        System.out.println("Prefix: " + convert(postfix));
    }
}
```

Explanation:

1. Traverse from left to right.
 2. Push operands.
 3. On operator, pop two, combine into prefix, push back.
-

PROBLEM: 30

Problem Statement:

Convert Postfix to Infix using Stack.

Java Code:

```
java
CopyEdit
import java.util.*;

public class PostfixToInfix {
    public static String convert(String expr) {
        Stack<String> stack = new Stack<>();
        for (int i = 0; i < expr.length(); i++) {
            char c = expr.charAt(i);
            if (Character.isLetterOrDigit(c)) {
                stack.push(c + "");
            } else {
                String op2 = stack.pop();
                String op1 = stack.pop();
                stack.push("(" + op1 + c + op2 + "");
            }
        }
        return stack.peek();
    }

    public static void main(String[] args) {
        String postfix = "ab+c*";
        System.out.println("Infix: " + convert(postfix));
    }
}
```

Explanation:

1. Push operands to stack.
 2. On operator, pop two operands, wrap them with operator in infix, and push back.
-

PROBLEM: 31

Problem Statement:

Convert an Infix expression to Prefix using Stack.

Java Code:

```
java
CopyEdit
import java.util.*;

public class InfixToPrefix {
    public static int precedence(char ch) {
        switch (ch) {
            case '+': case '-': return 1;
            case '*': case '/': return 2;
            case '^': return 3;
        }
        return -1;
    }

    public static String reverse(String expr) {
        StringBuilder sb = new StringBuilder(expr);
        sb.reverse();
        for (int i = 0; i < sb.length(); i++) {
            if (sb.charAt(i) == '(') sb.setCharAt(i, ')');
            else if (sb.charAt(i) == ')') sb.setCharAt(i, '(');
        }
        return sb.toString();
    }

    public static String convert(String infix) {
        String revInfix = reverse(infix);
        Stack<Character> stack = new Stack<>();
        StringBuilder result = new StringBuilder();

        for (char c : revInfix.toCharArray()) {
            if (Character.isLetterOrDigit(c)) {
                result.append(c);
            } else if (c == '(') {
                stack.push(c);
            } else if (c == ')') {
                while (!stack.isEmpty() && stack.peek() != '(')
                    result.append(stack.pop());
                stack.pop();
            } else {
                while (!stack.isEmpty() && precedence(c) <
precedence(stack.peek()))
                    result.append(stack.pop());
                stack.push(c);
            }
        }

        while (!stack.isEmpty())
            result.append(stack.pop());

        return result.reverse().toString();
    }
}
```

```

    }

    public static void main(String[] args) {
        String infix = "a+b*(c^d-e)^(f+g*h)-i";
        System.out.println("Prefix: " + convert(infix));
    }
}

```

Explanation:

1. Reverse infix and swap (with) .
2. Convert to postfix.
3. Reverse the result to get prefix.

Great! Continuing in your requested format from **Problem 32**:

PROBLEM: 32

Problem Statement:

Write a Java program to find the **Next Greater Element** (NGE) for each element in an array.

Java Code:

```

import java.util.*;

public class NextGreaterElement {
    public static int[] nextGreater(int[] nums) {
        Stack<Integer> stack = new Stack<>();
        int[] res = new int[nums.length];

        for (int i = nums.length - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() <= nums[i])
                stack.pop();
            res[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(nums[i]);
        }

        return res;
    }

    public static void main(String[] args) {
        int[] nums = {4, 5, 2, 10};
        System.out.println("NGE: " + Arrays.toString(nextGreater(nums)));
    }
}

```

Explanation:

1. Traverse array from right to left.

2. Use a stack to keep potential NGE.
 3. For each element, pop smaller elements from stack and assign NGE.
-

PROBLEM: 33

Problem Statement:

Write a Java program to find the **Next Greater Element II** (Circular Array).

Java Code:

```
import java.util.*;

public class NextGreaterElement2 {
    public static int[] nextGreaterElements(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = 2 * n - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() <= nums[i % n])
                stack.pop();
            if (i < n)
                res[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(nums[i % n]);
        }
        return res;
    }

    public static void main(String[] args) {
        int[] nums = {1, 2, 1};
        System.out.println("NGE II: " +
            Arrays.toString(nextGreaterElements(nums)));
    }
}
```

Explanation:

1. Traverse array twice for circular effect.
 2. Use modulo to simulate circular behavior.
 3. Stack stores next greater values.
-

PROBLEM: 34

Problem Statement:

Write a Java program to find the **Next Smaller Element** for each element in the array.

Java Code:

```
import java.util.*;

public class NextSmallerElement {
    public static int[] nextSmaller(int[] nums) {
        int[] res = new int[nums.length];
        Stack<Integer> stack = new Stack<>();

        for (int i = nums.length - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() >= nums[i])
                stack.pop();
            res[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(nums[i]);
        }

        return res;
    }

    public static void main(String[] args) {
        int[] nums = {4, 8, 5, 2, 25};
        System.out.println("Next Smaller Element: " +
            Arrays.toString(nextSmaller(nums)));
    }
}
```

Explanation:

1. Traverse from right to left.
 2. Use stack to keep smaller values.
 3. Pop greater values and assign the next smaller one.
-

PROBLEM: 35

Problem Statement:

Find **Number of Next Greater Elements** to the right for each element.

Java Code:

```
import java.util.*;

public class CountNGEsRight {
    public static int[] countNGEs(int[] nums) {
        int[] res = new int[nums.length];
        Stack<Integer> stack = new Stack<>();

        for (int i = nums.length - 1; i >= 0; i--) {
            while (!stack.isEmpty() && stack.peek() <= nums[i])
                stack.pop();
            res[i] = stack.size();
            stack.push(nums[i]);
        }
    }
}
```

```

        }

        return res;
    }

    public static void main(String[] args) {
        int[] nums = {4, 5, 2, 10};
        System.out.println("NGE Count: " +
Arrays.toString(countNGEs(nums)));
    }
}

```

Explanation:

1. Stack tracks greater elements.
 2. At each position, count how many greater values are right to it.
-

PROBLEM: 36

Problem Statement:

Trapping Rainwater problem: Find the amount of water that can be trapped.

Java Code:

```

public class TrappingRainWater {
    public static int trap(int[] height) {
        int n = height.length, left = 0, right = n - 1, leftMax = 0,
rightMax = 0, water = 0;
        while (left < right) {
            if (height[left] <= height[right]) {
                leftMax = Math.max(leftMax, height[left]);
                water += leftMax - height[left];
                left++;
            } else {
                rightMax = Math.max(rightMax, height[right]);
                water += rightMax - height[right];
                right--;
            }
        }
        return water;
    }

    public static void main(String[] args) {
        int[] height = {4,2,0,3,2,5};
        System.out.println("Trapped Water: " + trap(height));
    }
}

```

Explanation:

1. Two pointers from both ends.
2. Maintain max height from left and right.
3. Water at each index = $\min(\text{leftMax}, \text{rightMax}) - \text{height}[i]$.

PROBLEM:38

Problem Statement:

Write a Java program to calculate the **sum of minimums of all subarrays** of a given integer array.

Java Code:

```
java
CopyEdit
import java.util.*;

public class SubarrayMinimumSum {
    public static void main(String[] args) {
        int[] arr = {3, 1, 2, 4};
        System.out.println("Sum of subarray minimums: " +
sumSubarrayMins(arr));
    }

    public static int sumSubarrayMins(int[] arr) {
        int mod = 1_000_000_007;
        int n = arr.length;
        Stack<Integer> stack = new Stack<>();
        int[] prev = new int[n];
        int[] next = new int[n];

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && arr[stack.peek()] > arr[i])
stack.pop();
            prev[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(i);
        }

        stack.clear();

        for (int i = n - 1; i >= 0; i--) {
            while (!stack.isEmpty() && arr[stack.peek()] >= arr[i])
stack.pop();
            next[i] = stack.isEmpty() ? n : stack.peek();
            stack.push(i);
        }

        long result = 0;
        for (int i = 0; i < n; i++) {
            long count = (i - prev[i]) * (next[i] - i);
            result = (result + arr[i] * count) % mod;
        }
        return (int) result;
    }
}
```

Explanation:

1. For each element, we find how many subarrays it's the minimum for using monotonic stacks.
2. `prev[i]` gives the previous lesser element's index.
3. `next[i]` gives the next lesser or equal element's index.
4. Total subarrays where `arr[i]` is minimum: $(i - \text{prev}[i]) * (\text{next}[i] - i)$.
5. We multiply this count with `arr[i]` and add to result.

PROBLEM:39

Problem Statement:

Write a Java program to simulate the **asteroid collision** process. Each asteroid moves in the same direction and has a size. A positive value indicates the asteroid is moving right, and a negative value indicates it's moving left. Find the state of the asteroids after all collisions.

Java Code:

```
java
CopyEdit
import java.util.*;

public class AsteroidCollision {
    public static void main(String[] args) {
        int[] asteroids = {5, 10, -5};
        System.out.println("Asteroids after collision: " +
            Arrays.toString(asteroidCollision(asteroids)));
    }

    public static int[] asteroidCollision(int[] asteroids) {
        Stack<Integer> stack = new Stack<>();

        for (int ast : asteroids) {
            boolean alive = true;

            while (alive && ast < 0 && !stack.isEmpty() && stack.peek() > 0)
            {
                if (stack.peek() < -ast) {
                    stack.pop();
                } else if (stack.peek() == -ast) {
                    stack.pop();
                    alive = false;
                } else {
                    alive = false;
                }
            }

            if (alive) stack.push(ast);
        }

        int[] result = new int[stack.size()];
        for (int i = stack.size() - 1; i >= 0; i--) {
            result[i] = stack.pop();
        }
    }
}
```

```

    }

    return result;
}
}

```

Explanation:

1. Positive values move right, negative move left.
2. Collisions happen when a left-moving asteroid meets a right-moving one.
3. The stack maintains the "live" asteroids.
4. For each new asteroid, check collisions with top of stack.
5. Add it to stack if it survives.

PROBLEM:40

Problem Statement:

Write a Java program to find the **sum of subarray ranges**. The range of a subarray is the difference between its maximum and minimum element. Return the sum of all subarray ranges.

Java Code:

```

java
CopyEdit
public class SubarrayRangeSum {
    public static void main(String[] args) {
        int[] nums = {1, 2, 3};
        System.out.println("Sum of subarray ranges: " +
subArrayRanges(nums));
    }

    public static long subArrayRanges(int[] nums) {
        long sum = 0;
        int n = nums.length;

        for (int i = 0; i < n; i++) {
            int min = nums[i], max = nums[i];
            for (int j = i; j < n; j++) {
                min = Math.min(min, nums[j]);
                max = Math.max(max, nums[j]);
                sum += max - min;
            }
        }

        return sum;
    }
}

```

Explanation:

1. Iterate over all subarrays using two loops.

2. Track min and max within each subarray.
3. Add (max - min) to sum for each subarray.

PROBLEM:41

Problem Statement:

Write a Java program to **remove k digits** from a number string to make the smallest possible number.

Java Code:

```
java
CopyEdit
import java.util.*;

public class RemoveKDigits {
    public static void main(String[] args) {
        String num = "1432219";
        int k = 3;
        System.out.println("Smallest number: " + removeKdigits(num, k));
    }

    public static String removeKdigits(String num, int k) {
        Stack<Character> stack = new Stack<>();

        for (char c : num.toCharArray()) {
            while (!stack.isEmpty() && k > 0 && stack.peek() > c) {
                stack.pop();
                k--;
            }
            stack.push(c);
        }

        while (k-- > 0) stack.pop();

        StringBuilder sb = new StringBuilder();
        for (char c : stack) sb.append(c);

        // Remove leading zeros
        while (sb.length() > 1 && sb.charAt(0) == '0') sb.deleteCharAt(0);

        return sb.length() == 0 ? "0" : sb.toString();
    }
}
```

Explanation:

1. Use a stack to build the number.
 2. Remove digits that are larger than the current digit.
 3. Finally, remove leading zeros.
-

PROBLEM:42

Problem Statement:

Write a Java program to find the **largest rectangle in a histogram**. Given an array of bar heights, find the area of the largest rectangle.

Java Code:

```
java
CopyEdit
import java.util.*;

public class LargestRectangleHistogram {
    public static void main(String[] args) {
        int[] heights = {2, 1, 5, 6, 2, 3};
        System.out.println("Max rectangle area: " +
largestRectangleArea(heights));
    }

    public static int largestRectangleArea(int[] heights) {
        Stack<Integer> stack = new Stack<>();
        int maxArea = 0, i = 0;
        int n = heights.length;

        while (i < n) {
            if (stack.isEmpty() || heights[i] >= heights[stack.peek()]) {
                stack.push(i++);
            } else {
                int height = heights[stack.pop()];
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                maxArea = Math.max(maxArea, height * width);
            }
        }

        while (!stack.isEmpty()) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }

        return maxArea;
    }
}
```

Explanation:

1. Use stack to track increasing heights.
 2. For each bar, calculate area with the bar as the smallest height.
 3. Track max area found.
-

PROBLEM:43

Problem Statement:

Write a Java program to find the **maximal rectangle** in a binary matrix of 0s and 1s. Return the area of the largest rectangle containing only 1s.

Java Code:

```
java
CopyEdit
public class MaximalRectangle {
    public static void main(String[] args) {
        char[][] matrix = {
            {'1', '0', '1', '0', '0'},
            {'1', '0', '1', '1', '1'},
            {'1', '1', '1', '1', '1'},
            {'1', '0', '0', '1', '0'}
        };
        System.out.println("Maximal rectangle area: " +
            maximalRectangle(matrix));
    }

    public static int maximalRectangle(char[][] matrix) {
        if (matrix.length == 0) return 0;
        int maxArea = 0;
        int[] heights = new int[matrix[0].length];

        for (char[] row : matrix) {
            for (int j = 0; j < row.length; j++) {
                heights[j] = row[j] == '1' ? heights[j] + 1 : 0;
            }
            maxArea = Math.max(maxArea, largestRectangleArea(heights));
        }

        return maxArea;
    }

    public static int largestRectangleArea(int[] heights) {
        Stack<Integer> stack = new Stack<>();
        int maxArea = 0, i = 0;
        int n = heights.length;

        while (i < n) {
            if (stack.isEmpty() || heights[i] >= heights[stack.peek()]) {
                stack.push(i++);
            } else {
                int height = heights[stack.pop()];
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                maxArea = Math.max(maxArea, height * width);
            }
        }

        while (!stack.isEmpty()) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }

        return maxArea;
    }
}
```

```
}  
}
```

Explanation:

1. Convert each row of matrix into histogram.
2. Use Largest Rectangle in Histogram logic row-wise.
3. Track max area.

PROBLEM:44

Problem Statement:

Write a Java program to find the **Sliding Window Maximum**. Given an array `nums` and an integer `k`, find the maximum for each sliding window of size `k`.

Java Code:

```
java  
CopyEdit  
import java.util.*;  
  
public class SlidingWindowMaximum {  
    public static void main(String[] args) {  
        int[] nums = {1,3,-1,-3,5,3,6,7};  
        int k = 3;  
        int[] result = maxSlidingWindow(nums, k);  
        System.out.println(Arrays.toString(result));  
    }  
  
    public static int[] maxSlidingWindow(int[] nums, int k) {  
        Deque<Integer> dq = new ArrayDeque<>();  
        int[] res = new int[nums.length - k + 1];  
        int i = 0;  
  
        for (int j = 0; j < nums.length; j++) {  
            while (!dq.isEmpty() && dq.peekFirst() <= j - k) dq.pollFirst();  
            while (!dq.isEmpty() && nums[dq.peekLast()] < nums[j])  
                dq.pollLast();  
            dq.offerLast(j);  
            if (j >= k - 1) res[i++] = nums[dq.peekFirst()];  
        }  
  
        return res;  
    }  
}
```

Explanation:

1. Use deque to keep track of window max indices.
2. Maintain window of size `k`.
3. Front of deque always holds the index of maximum element.

PROBLEM:45

Problem Statement:

Write a Java program to solve the **Stock Span Problem**. For each day, calculate how many consecutive previous days the stock price was less than or equal to the current price.

Java Code:

```
java
CopyEdit
import java.util.*;

public class StockSpan {
    public static void main(String[] args) {
        int[] prices = {100, 80, 60, 70, 60, 75, 85};
        int[] result = calculateSpan(prices);
        System.out.println(Arrays.toString(result));
    }

    public static int[] calculateSpan(int[] prices) {
        Stack<Integer> stack = new Stack<>();
        int n = prices.length;
        int[] span = new int[n];

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && prices[stack.peek()] <= prices[i]) {
                stack.pop();
            }
            span[i] = stack.isEmpty() ? i + 1 : i - stack.peek();
            stack.push(i);
        }

        return span;
    }
}
```

Explanation:

1. Use stack to maintain indices of useful elements.
 2. If stack is empty, it means price is highest so far.
 3. Else, subtract the last greater price index.
-

PROBLEM:46

Problem Statement:

Write a Java program to find the **Celebrity** at a party. A celebrity is someone who is known by everyone but knows no one.

Java Code:

```
java
CopyEdit
public class CelebrityProblem {
    static int MATRIX[][] = {
        {0, 1, 1},
        {0, 0, 1},
        {0, 0, 0}
    };

    public static void main(String[] args) {
        System.out.println("Celebrity Index: " + findCelebrity(3));
    }

    static boolean knows(int a, int b) {
        return MATRIX[a][b] == 1;
    }

    public static int findCelebrity(int n) {
        int candidate = 0;
        for (int i = 1; i < n; i++) {
            if (knows(candidate, i)) {
                candidate = i;
            }
        }

        for (int i = 0; i < n; i++) {
            if (i != candidate && (knows(candidate, i) || !knows(i,
candidate)))) {
                return -1;
            }
        }

        return candidate;
    }
}
```

Explanation:

1. Use elimination to find candidate.
 2. Verify if candidate is known by everyone but knows no one.
-

PROBLEM:47**Problem Statement:**

Write a Java program to implement an **LRU (Least Recently Used) Cache** using LinkedHashMap.

Java Code:

```

java
CopyEdit
import java.util.*;

class LRUCache extends LinkedHashMap<Integer, Integer> {
    private int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // accessOrder = true
        this.capacity = capacity;
    }

    public int get(int key) {
        return super.getDefault(key, -1);
    }

    public void put(int key, int value) {
        super.put(key, value);
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest)
    {
        return size() > capacity;
    }

    public static void main(String[] args) {
        LRUCache cache = new LRUCache(2);
        cache.put(1, 1);
        cache.put(2, 2);
        System.out.println(cache.get(1)); // 1
        cache.put(3, 3);
        System.out.println(cache.get(2)); // -1
    }
}

```

Explanation:

1. Extends `LinkedHashMap` to override eviction logic.
 2. Uses access order to track recently used items.
 3. Evicts eldest entry when size exceeds capacity.
-

PROBLEM:48

Problem Statement:

Write a Java program to implement **LFU (Least Frequently Used) Cache**.

Java Code:

```

java
CopyEdit

```

```

import java.util.*;

class LFUCache {
    class Node {
        int key, val, freq;
        Node(int k, int v) {
            key = k;
            val = v;
            freq = 1;
        }
    }

    private int capacity, minFreq;
    private Map<Integer, Node> keyMap;
    private Map<Integer, LinkedHashSet<Integer>> freqMap;

    public LFUCache(int capacity) {
        this.capacity = capacity;
        keyMap = new HashMap<>();
        freqMap = new HashMap<>();
    }

    public int get(int key) {
        if (!keyMap.containsKey(key)) return -1;
        Node node = keyMap.get(key);
        updateFreq(node);
        return node.val;
    }

    public void put(int key, int value) {
        if (capacity == 0) return;

        if (keyMap.containsKey(key)) {
            Node node = keyMap.get(key);
            node.val = value;
            updateFreq(node);
        } else {
            if (keyMap.size() == capacity) {
                int evict = freqMap.get(minFreq).iterator().next();
                freqMap.get(minFreq).remove(evict);
                keyMap.remove(evict);
            }
            Node node = new Node(key, value);
            keyMap.put(key, node);
            minFreq = 1;
            freqMap.computeIfAbsent(1, x -> new LinkedHashSet<>()).add(key);
        }
    }

    private void updateFreq(Node node) {
        int freq = node.freq;
        freqMap.get(freq).remove(node.key);
        if (freqMap.get(freq).isEmpty() && freq == minFreq) minFreq++;
        node.freq++;
        freqMap.computeIfAbsent(node.freq, x -> new
LinkedHashSet<>()).add(node.key);
    }

    public static void main(String[] args) {
        LFUCache cache = new LFUCache(2);
        cache.put(1, 1);
    }
}

```

```
        cache.put(2, 2);
        System.out.println(cache.get(1)); // 1
        cache.put(3, 3);
        System.out.println(cache.get(2)); // -1
        System.out.println(cache.get(3)); // 3
    }
}
```

Explanation:

1. Tracks frequency and recency of usage.
2. Uses two maps: keyMap (stores key to node), freqMap (stores freq to keys).
3. When capacity is full, removes least frequently used key.

Congratulations on completing the PDF created by SYNTAX ERROR, also known as Abhishek Rathor! Your dedication and perseverance are truly commendable. For more insights and updates, feel free to connect on Instagram at SYNTAX ERROR. Keep up the great work!